

 Marwadi University	Marwadi University Faculty of Technology Department of Information and Communication Technology	
Subject: Artificial Intelligence (01CT0616)	Aim: To obtain the best fit line over single feature scattered datapoints using Linear Regression	
Experiment No: 02	Date:	Enrollment No:92410133034

Aim: To obtain the best fit line over single feature scattered datapoints using Linear Regression

IDE: Google Colab

Theory:

Linear regression is a method for determining the best linear relationship between two variables X and Y . If variables X and Y are uncorrelated, it is pointless embarking upon linear regression. However, if a reasonable degree of correlation exists between X and Y then linear regression may be a useful means to describe the relationship between the two variables. The usual approach is to use the *least-squares* method, which minimizes the squared difference between the actual data points and a straight line. Let $[x_i, y_i], i = 1, 2, 3, \dots, N$ be the N pairs of data values of the variables X and Y . The straight-line relating X and Y is $y = mx + c$, where m and c are the gradient and constant values (to be determined) defining the straight line. Thus, $y(x_i) - y_i$ is the difference between the line and data point i (see Fig. 1). Taking all the data points, we seek values of m and c that minimize the squared difference SD .

$$\sum_1^N [y(x_i) - y_i]^2$$

This is achieved by calculating the partial derivatives of SD with respect to m and c and finding the pair $[m, c]$ such that SD is at a minimum.

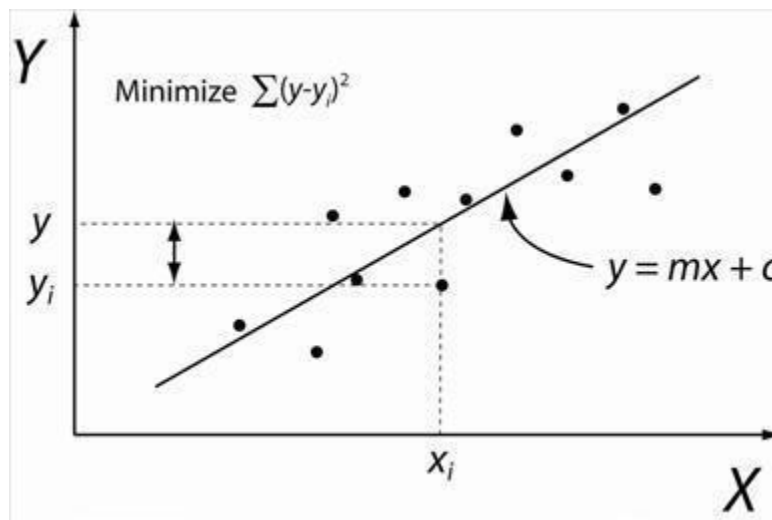


Figure 1: Illustration of Linear Regression. Linear least squares regression, the idea is to find the line $y = mx + c$ that minimizes the mean squared difference between the line and the data points

 Marwadi University	Marwadi University Faculty of Technology Department of Information and Communication Technology	
Subject: Artificial Intelligence (01CT0616)	Aim: To obtain the best fit line over single feature scattered datapoints using Linear Regression	
Experiment No: 02	Date:	Enrollment No:92410133034

Batch Gradient Descent:

Gradient Descent is an optimization algorithm used for minimizing the cost function in various machine learning algorithms. It is basically used for updating the parameters of the learning model. Batch gradient descent which processes all the training examples for each iteration of gradient descent. But if the number of training examples is large, then batch gradient descent is computationally very expensive.

Let m be the number of training examples. Let n be the number of features.

Algorithm for batch gradient descent :

Let $h_{\theta}(x)$ be the hypothesis for linear regression. Then, the cost function is given by:

Let Σ represents the sum of all training examples from $i=1$ to m .

$$J_{\text{train}}(\theta) = (1/2m) \Sigma (h_{\theta}(x^{(i)}) - y^{(i)})^2$$

Repeat {

$$\theta_j = \theta_j - (\text{learning rate}/m) * \Sigma (h_{\theta}(x^{(i)}) - y^{(i)})x_j^{(i)}$$

For every $j=0 \dots n$

}

Where $x_j^{(i)}$ Represents the j^{th} feature of the i^{th} training example. So if m is very large (e.g. 5 million training samples), then it takes hours or even days to converge to the global minimum. That's why for large datasets, it is not recommended to use batch gradient descent as it slows down the learning.

Pre Lab Exercise:

- Explain the meaning of linear regression.

Linear regression is a method to find the relationship between two variables using a straight line

- Write three applications of linear regression

- House price prediction
- Sales prediction
- Temperature forecasting

- Write three advantages of linear regression

- Simple and easy to use
- Fast computation
- Easy to interpret

- Write three limitations of linear regression

- Works only for linear data
- Sensitive to outliers
- Assumes relationship between variables

 Marwadi University	Marwadi University Faculty of Technology Department of Information and Communication Technology	
Subject: Artificial Intelligence (01CT0616)	Aim: To obtain the best fit line over single feature scattered datapoints using Linear Regression	
Experiment No: 02	Date:	Enrollment No:92410133034

Methodology:

1. Load the basic libraries and packages

```
[8]
✓ 0s
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
```

2. Load the dataset

```
[9]
✓ 0s
# Create simple dataset (single feature)

data = pd.DataFrame({
    'X': [1, 2, 3, 4, 5],
    'Y': [2, 4, 5, 4, 5]
})

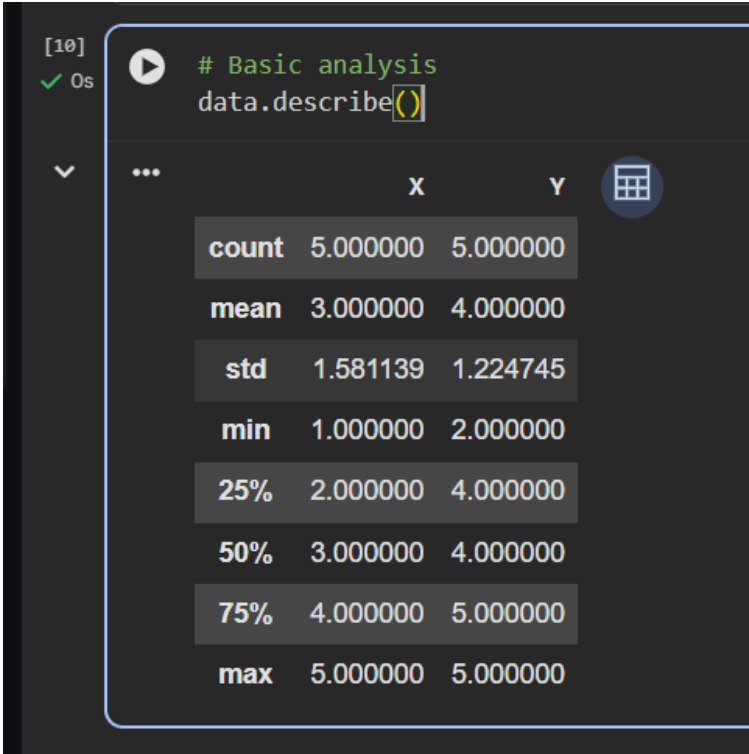
data
```

	X	Y
0	1	2
1	2	4
2	3	5
3	4	4
4	5	5

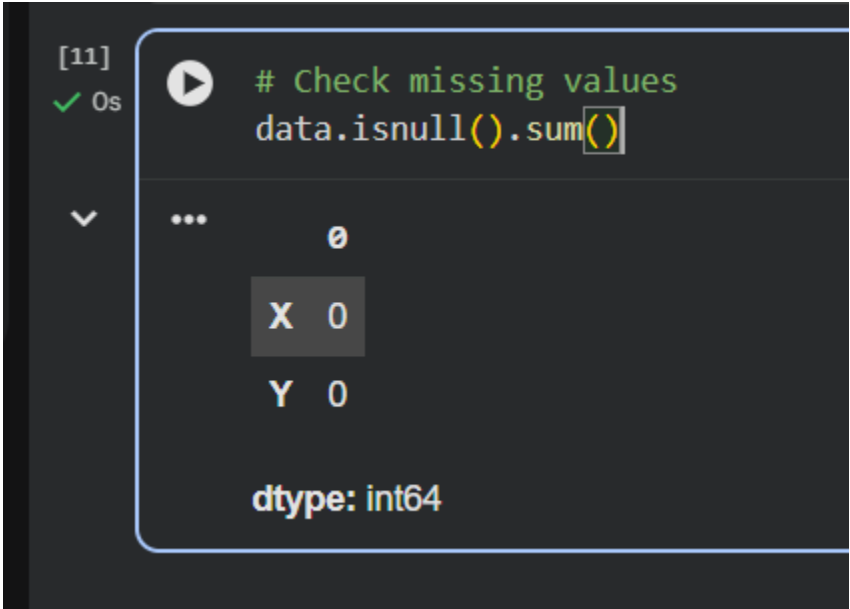
Next steps: [Generate code with data](#) [New interactive sheet](#)

 Marwadi University	Marwadi University Faculty of Technology Department of Information and Communication Technology	
Subject: Artificial Intelligence (01CT0616)	Aim: To obtain the best fit line over single feature scattered datapoints using Linear Regression	
Experiment No: 02	Date:	Enrollment No:92410133034

3. Analyse the dataset

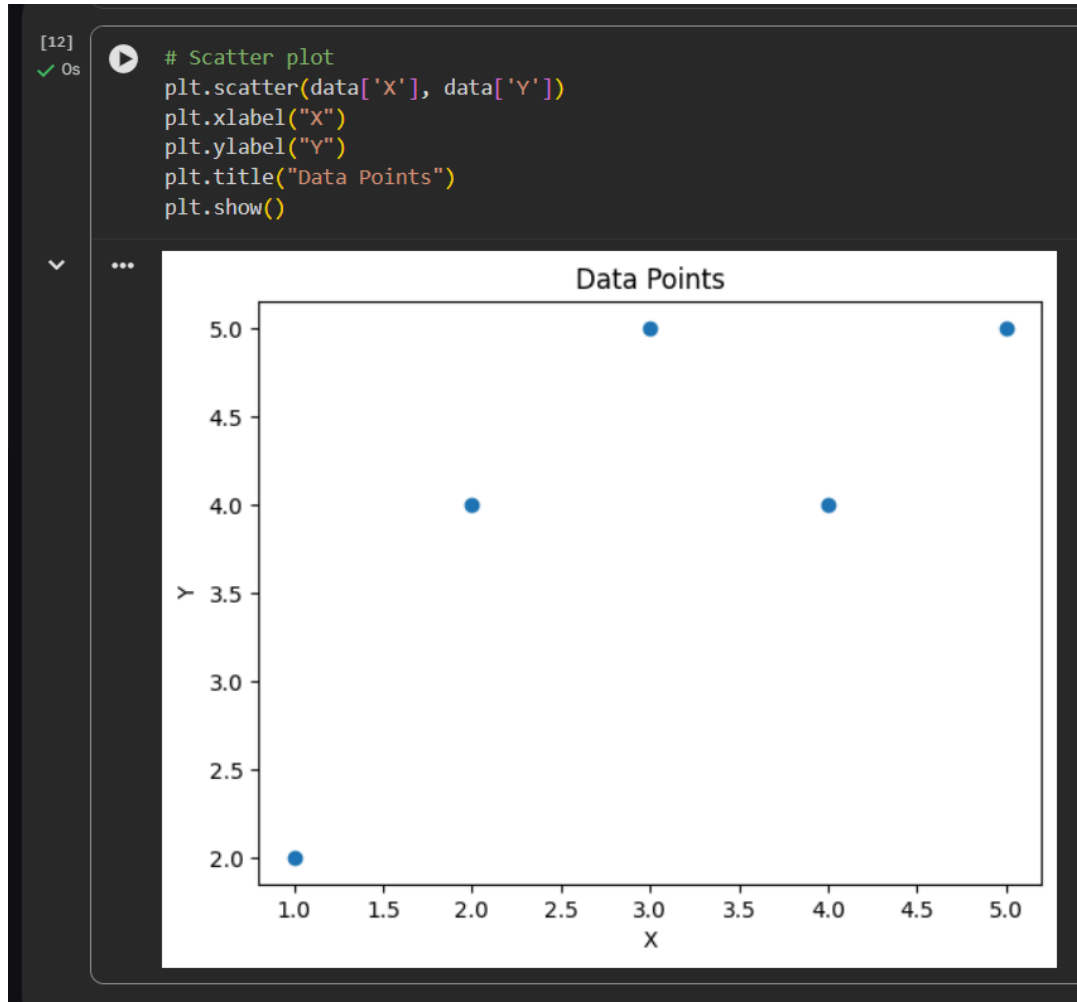


4. Pre-process the data



 Marwadi University	Marwadi University Faculty of Technology Department of Information and Communication Technology	
Subject: Artificial Intelligence (01CT0616)	Aim: To obtain the best fit line over single feature scattered datapoints using Linear Regression	
Experiment No: 02	Date:	Enrollment No:92410133034

5. Visualize the Data



6. Separate the feature and prediction value columns

```
[13]
✓ 0s # Separate X and Y

X = data['X'].values
Y = data['Y'].values

X, Y

... (array([1, 2, 3, 4, 5]), array([2, 4, 5, 4, 5]))
```

 Marwadi University	Marwadi University Faculty of Technology Department of Information and Communication Technology	
Subject: Artificial Intelligence (01CT0616)	Aim: To obtain the best fit line over single feature scattered datapoints using Linear Regression	
Experiment No: 02	Date:	Enrollment No:92410133034

7. Write the Hypothesis Function

```
[14]
✓ 0s # Hypothesis function:  $y = m \cdot x + c$ 

def hypothesis(x, m, c):
    return m * x + c
```

8. Write the Cost Function

```
[15]
✓ 0s # Cost function (Mean Squared Error)

def cost_function(x, y, m, c):
    n = len(x)
    total_error = 0

    for i in range(n):
        total_error += (hypothesis(x[i], m, c) - y[i])**2

    return total_error / (2 * n)
```

9. Write the Gradient Descent optimization algorithm

```
[16]
✓ 0s # Gradient Descent

def gradient_descent(x, y, m, c, learning_rate, iterations):
    n = len(x)

    for _ in range(iterations):
        dm = 0
        dc = 0

        for i in range(n):
            error = hypothesis(x[i], m, c) - y[i]
            dm += error * x[i]
            dc += error

        m = m - (learning_rate * dm / n)
        c = c - (learning_rate * dc / n)

    return m, c
```

 Marwadi University	Marwadi University Faculty of Technology Department of Information and Communication Technology	
Subject: Artificial Intelligence (01CT0616)	Aim: To obtain the best fit line over single feature scattered datapoints using Linear Regression	
Experiment No: 02	Date:	Enrollment No:92410133034

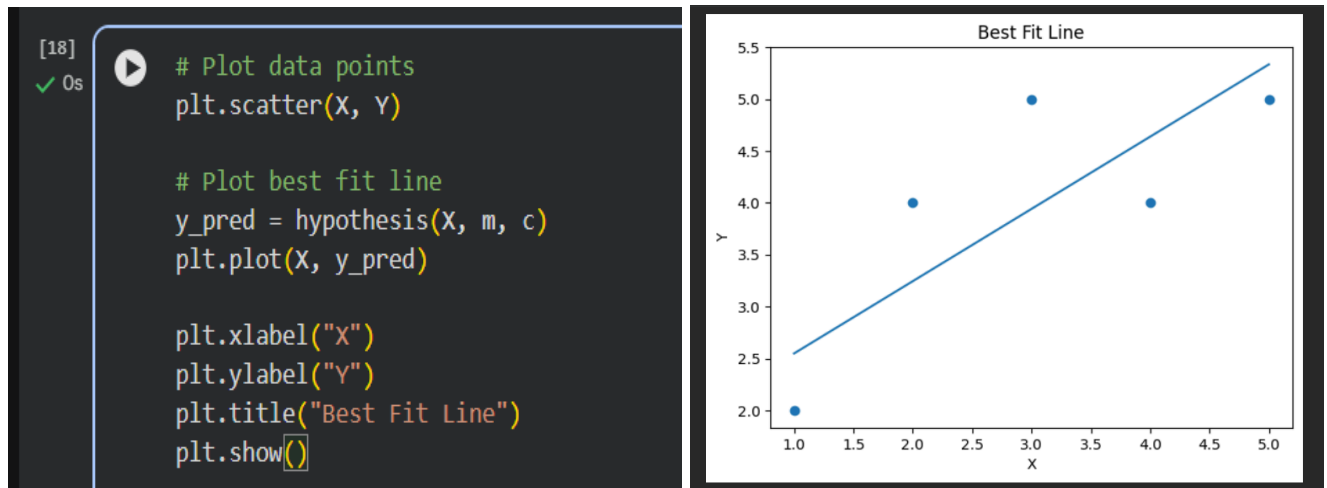
10. Apply the training over the dataset to minimize the loss

```
[17]
✓ Os
# Initialize values
m = 0
c = 0
learning_rate = 0.01
iterations = 1000

# Train model
m, c = gradient_descent(X, Y, m, c, learning_rate, iterations)

m, c
... (np.float64(0.6963550004885218), np.float64(1.8521278749603411))
```

11. Find the best fit line to the given dataset



12. Observe the cost function vs iterations learning curve

 Marwadi University	Marwadi University Faculty of Technology Department of Information and Communication Technology	
Subject: Artificial Intelligence (01CT0616)	Aim: To obtain the best fit line over single feature scattered datapoints using Linear Regression	
Experiment No: 02	Date:	Enrollment No:92410133034

```

[19]
✓ Os
# Cost vs iterations

def gradient_descent_with_cost(x, y, m, c, learning_rate, iterations):
    n = len(x)
    cost_list = []

    for _ in range(iterations):
        dm = 0
        dc = 0

        for i in range(n):
            error = hypothesis(x[i], m, c) - y[i]
            dm += error * x[i]
            dc += error

        m = m - (learning_rate * dm / n)
        c = c - (learning_rate * dc / n)

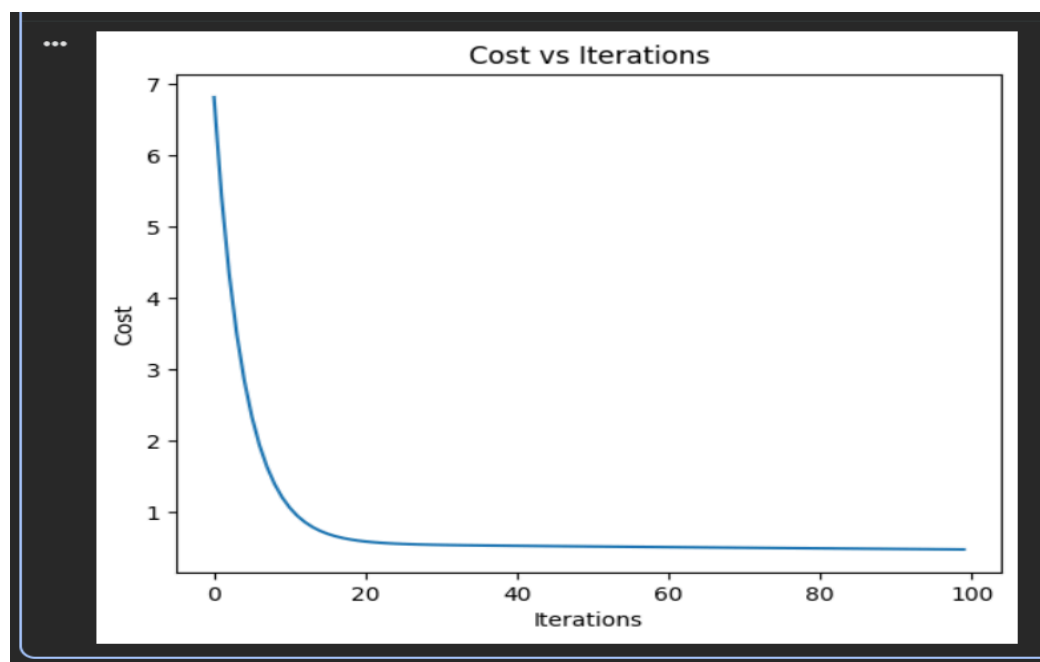
        cost_list.append(cost_function(x, y, m, c))

    return m, c, cost_list

# Run again to get cost values
m, c, cost_list = gradient_descent_with_cost(X, Y, 0, 0, 0.01, 100)

# Plot cost vs iterations
plt.plot(cost_list)
plt.xlabel("Iterations")
plt.ylabel("Cost")
plt.title("Cost vs Iterations")
plt.show()

```



 Marwadi University	Marwadi University Faculty of Technology Department of Information and Communication Technology	
Subject: Artificial Intelligence (01CT0616)	Aim: To obtain the best fit line over single feature scattered datapoints using Linear Regression	
Experiment No: 02	Date:	Enrollment No:92410133034

Results:

To be attached with

- Datapoints scattering (without best fit line)
Scatter plot shows distribution of data points between X and Y.
- Scatter plot showing the Best fit line in the last iteration of training
Best fit line is plotted over data points showing linear relationship after training.

Observation and Result Analysis:

- Nature of the dataset
Dataset shows a linear relationship between X and Y.
- During Training Process
Cost decreases with iterations and model performance improves.
- After the training Process
Model finds best fit line with optimal values of m and c.

Post Lab Exercise:

- What are the major assumptions considered in linear regression
Linear relationship, no multicollinearity, constant variance, and normal distribution of errors.
- Why MSE is used instead of MAE for calculating the loss function
MSE gives more weight to large errors and is easier to optimize mathematically.
- How can the behaviour of outliers be understood while dealing with the unseen dataset
Outliers can be detected using IQR or plots and they can affect model accuracy.
- Derive the Normal Equation for the Linear Regression.
Normal equation directly calculates parameters using matrix formula without iteration.